

Reviewer Pack — Minimal Rank of Algebraic Cycles in Characteristic p over $Af...$

Entry 8c339426d072 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 23:23:29 UTC

Minimal Rank of Algebraic Cycles in Characteristic p over Affine Schemes vs Quantifier Depth for k -CNF

Entry ID: 8c339426d072 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 23:23:29 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Characteristic p Theory) **Field B** (complexity object): Complexity Theory: Quantifier Depth for k -CNF

Statement:

[‘For any affine scheme X defined over a field with characteristic p , the minimal rank of an algebraic cycle in the Chow ring of X is polynomially bounded by the quantifier depth of a refutation proof for k -CNF instances.’, ‘Equivalently, for all $n \leq 40$, if a k -CNF instance has a quantifier depth d , then there exists an affine scheme over a field with characteristic p such that the minimal rank of an algebraic cycle in its Chow ring is at most Cd for some constant C .’, ‘Further, this relation holds for all instances with property P such that property Q holds.’]

Rationale (proposer’s reasoning):

[‘Algebraic cycles provide a rich algebraic structure that can encode complex properties of geometric objects. Characteristic p theory offers a way to study these structures in a more robust manner, which could potentially reveal hidden complexity-theoretic properties.’, ‘Quantifier depth is a known measure of the difficulty of refutation proofs for k -CNF instances. If the ranks of algebraic cycles are polynomially bounded by this measure, it suggests that geometric structures might play a role in understanding proof complexity.’, ‘This bridge could expose a previously unexplored connection between algebraic geometry and proof complexity, potentially leading to new insights into the nature of computational hardness.’]

Taxonomy category: CHARACTERISTIC_P_ALGEBRAIC_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 915ed16d20bc9129

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the minimal rank of an algebraic cycle in the Chow ring of affine schemes over fields with characteristic p is at most Cd for all k -CNF instances with quantifier depth d , where C is a constant polynomially bounded by n and the property P . It is falsified if any instance with quantifier depth d has a corresponding scheme with an algebraic cycle rank greater than Cd .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def quantifier_depth(kcnf):
        # Placeholder for actual quantifier depth calculation
        return len(kcnf.split())

    def construct_scheme(n, d):
        # Placeholder for constructing an affine scheme
        # This is a dummy implementation and should be replaced with actual logic
        return n * d

    def minimal_rank(scheme):
        # Placeholder for computing the minimal rank of an algebraic cycle
        # This is a dummy implementation and should be replaced with actual logic
        return len(scheme)

    results = []
    for _ in range(30): # Ensure at least 30 instances per seed
        n = random.randint(5, 40)
        kcnf = ''.join(random.choices('01', k=n * 2)) # Dummy k-CNF instance
        d = quantifier_depth(kcnf)
        scheme = construct_scheme(n, d)
        rank = minimal_rank(scheme)
```

```

        results.append((n, d, rank))

metric_value = sum(rank for _, _, rank in results) / len(results)
conjecture_holds = all(rank <= 2 * n for _, _, rank in results) # Dummy bound
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Minimal Rank of Algebraic Cycles",
    "metric_value": metric_value,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = (sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results)) ** 0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fae4838d.py", line 66, in <module>
    trial_result = run_trial(seed)
                  ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fae4838d.py", line 41, in run_trial
    rank = minimal_rank(scheme)
          ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fae4838d.py", line 33, in minimal_rank

```

```
return len(scheme)
^^^^^^^^^^
```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18228
2	preregistration	ollama_remote	glm4:latest	0	0	10999
3	novelty	ollama_remote	glm4:latest	0	0	8457
4	novelty	ollama_remote	glm4:latest	0	0	8645
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13732
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8233
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9466
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8857
9	judge	ollama_remote	glm4:latest	0	0	11502

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98118 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8c339426d072.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8c339426d072.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8c339426d072.tar.gz` (if generated)